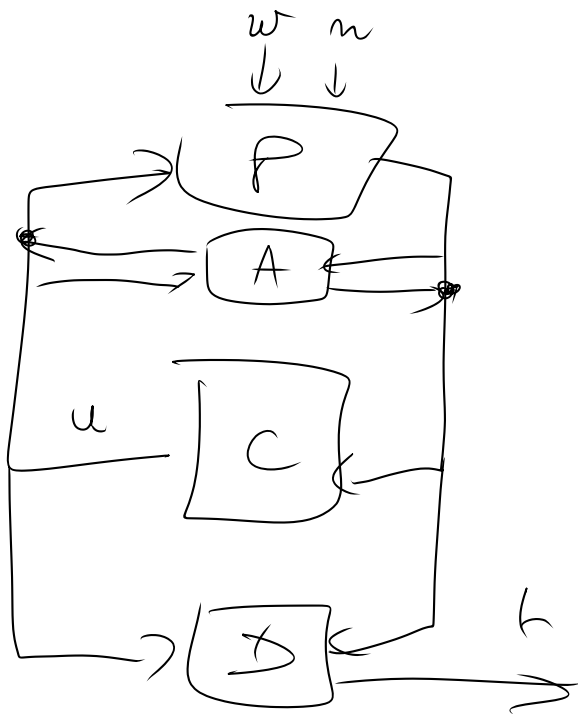


Today we take a look at a general discussion of detectability of attacks, as well as identification.

We will see that some attacks cannot be detected by any monitor. We will also discuss about the idea of consistent monitor. All these ideas are closely related.

Let's start from a reference model. Since we want to develop very general ideas, we have to take the simplest possible model, simply named "Simple Model".



The model for the plant was given by:

$$\begin{cases} \dot{x} = Ax + Bu + Gw \\ y = Cx + Du + Tn \end{cases}$$

In the model considered by the paper, we don't take into consideration the presence of disturbance and measurement noise.

In the middle we have, once again, the attacker.

Typically, the noise is modelled as stocastic variables. By removing these two, we remain with a deterministic system.

$$\begin{cases} \dot{x} = Ax + \cancel{Bu} + \cancel{Gw} \\ y = Cx + \cancel{Du} + \cancel{Tn} \end{cases}$$

The second simplification is given by ignoring Bu and Du .

$$\begin{cases} \dot{x} = Ax \\ y = Cx \end{cases}$$

Let's discuss briefly in the superimposition principle in linear systems.

We've seen that the formula for the solution we given by the free evolution part, depending on the initial state (x_0), plus the forced evolution, which was basically the integral of control. The same goes for $y(t)$.

What is then the super imposition principle?

Imagine doing one experiment: We take a system in a given initial state x_0 , and we apply one input u^1 . Call the output y^1 , and the state we get x^1 .

Then, proceede with another experiment: We take the same system, with initial state x_0 , and we apply another input, named u^2 . What we get is x^2 and y^2 .

If we do a third experiment, in which we apply the sum of inputs (for any linear combination), what we get in linear systems is that, by inputting $u^1 + u^2$, we get $x^1 + x^2$ and $y^1 + y^2$.

Let's apply this principle to our problem.

The reference model is given by:

$$\begin{cases} E\dot{x} = Ax + Bu \\ y = Cx + Du \end{cases}$$

A big warning: here u is the attack signal, not the control, which is, as we said before, ignored.

We face a powerful attacker that can corrupt any state variable and any measured output of the system.

We ignore the controls (computed by the controller), thus we can write the model with the controls like this:

$$\begin{cases} E\dot{x} = Ax + Bu + B_c u_c \\ y = Cx + Du + D_c u_c \end{cases}$$

With the values with $_c$ are the ones computed by the controller.

Bu and Du could be considered as input 1, decided by the attacker, and the values sent by the controls are named input 2.

Now, input 1 is unknown to the detector, while the values of input 2 are known. In fact, we can detect the effect of this known input. We can remove (subtract) this from the output. Thus, we focus on the model without actual control values.

Remember that the E matrix describes a descriptor system. Typically, E is singular:

$$E : \begin{pmatrix} * \dots * \\ 0 \dots 0 \\ \dots \\ 0 \dots 0 \end{pmatrix}$$

The lower part models algebraic constraints.

Swing equations: model generators, with differential equations, while with power flow equations we have algebraic equations.

The reference scheme is thus given by a system model as descriptor system

$$u(t) \longrightarrow \boxed{\begin{cases} E\dot{x} = Ax + Bu \\ y = Cx + Du \end{cases}} \\ \downarrow y(t) \\ \boxed{\text{monitor}}$$

We have an attack $u(t)$.

If the detector knows how the controller works, and has access to the measurements, he can internally compute the controls, without the need of passing to it the actual controls.

We once again consider the most powerful attacker, which is the one who can attack independently every state and every measurement. But why do we focus on him? If we're able to develop results with the most powerful attack, in reality we'll be fine, as we'll face a less powerful attack. This is done in Risk Analysis, to see what happens in the worst case.

Let's focus now on Bu and Du , the part modelling the attack.

B has a structure like this $B = [I \quad 0]$ and $D = [0 \quad I]$, and $u = \begin{pmatrix} u_x \\ u_y \end{pmatrix}$.

In general we have n state variables, so $u_x \in \mathbb{R}^n$, as well as $u_y \in \mathbb{R}^m$, with m being the amount of measured outputs.

We'll have systems like this:

$$\begin{cases} E\dot{x}_1 = Ax + u_{x,1} \\ E\dot{x}_2 = Ax + u_{x,2} \\ \vdots \\ \text{up to } u_{x,n} \end{cases}$$

And similarly for y .

Let's define n_x to be the number of state variables, and n_y the number of state outputs. Thus, in total the attack can corrupt $n_x + n_y$ variables.

Let's define K to be the attack set. It's the set of channels/variables that are

actually attacked by the attacker, $K \subseteq \{1, 2, 3, \dots, n_x + n_y\}$.

We have that $u_i(t) \neq 0$ for some t if $i \in K$ (if there is, some time, in which the attacker inject something in a channel). Similarly, $u_i(t) = 0 \forall t$ if $i \notin K$ (the attacker never injects anything into the channel).

B_K and D_K are the matrices in which we only consider the columns which are in the attack set. They take the name of attack signature.

≡ Example

$$B_K = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}$$

Similarly, u_K is the subvector of u indexed in K .

≡ Example above

$$u_K = \begin{pmatrix} u_{x_1} \\ u_{x_3} \end{pmatrix}$$

So, the attacker is composed by two things:

1. The attack set, so which channels to attack;
2. The attack logic. Remember that $u_K(t)$ is in function of time, and at any time the attacker must decide the specific attack signals to inject.

Whenever we study an attack, we look at least at these two things.

Let's complete the scheme:

$$\begin{array}{c} \downarrow u_k \\ \boxed{\begin{cases} E\dot{x} = Ax + Bu \\ y = Cx + Du \\ x_0 \text{ known} \end{cases}} \\ \downarrow y(t) = y(t, x_0, u_{K,[0,t]}) = y(t, x_0, u_K) \\ \boxed{\Lambda = \{E, A, C, y(t) \forall t\}} \begin{array}{l} \longrightarrow \Phi_1 \\ \longrightarrow \Phi_2 \end{array} \end{array}$$

With $t_0 = 0$. The $y(t, x_0, u_K)$ is a shorter notation, but it's important to remember that the "control" is in function of time.

Now, let's see the monitor. It's a deterministic algorithm in charge of:

1. Detection: at any time, understand if there is an attack or not. It's basically a yes or no problem, with Φ_1 being the first output of the monitor, the one related to detection. With $\Phi_1=\text{True}$, the monitor thinks that there is an attack (but it's not necessarily true). With $\Phi_1=\text{False}$, the monitor thinks there is no attack (but, as above, it's not necessarily true).
2. Identification: Understand what is the attack set K . The second output of the monitor Φ_2 is a set, which we call $\hat{K}$, so an estimation of the attacked set. We have that $\hat{K} \subseteq \{1, 2, \dots, n_x + n_y\}$, so it's the set of channels the monitor thinks they're under attack.

What are, now, the inputs of the monitor? They're given by:

- the measured outputs $y(t)$, $\forall t$
- The model of the system, which is known. This means matrices E, A, C . The monitor does not obviously know K , and it's job is to estimate it.

So, $\Lambda = \{E, A, C, y(t) \forall t > 0\}$.

If K is empty $\forall t$, there is no attack going. So the first problem is to determine whether K is empty or not, and after there is the trigger for an alarm we want to know the estimate of K , $\hat{K}$.

Now, let's study the property of consistency.

Monitors are algorithms designed by the operator of the system, just like the controller. We need to define, when designing a monitor, some consistency properties, which are given to ensure that we get a meaningful output from the monitor. They depend on the application and modelling assumptions.

For example, the trivial monitor that always outputs $\Phi_1=\text{True}$. What are the properties of this monitor? We have 100% detection rate, as it's always reporting true. We also have 100% false positive alarm rate.

We'll discuss three rules (consistency properties) necessary to limit as much as possible false positives. If we don't fix these minimal rules, we get something we cannot trust.

Before, some definitions:

- A nonzero attack ($u_k, K \neq \{0\}$) is detected $\iff \Phi_1=\text{True}$.
- A nonzero attack is identified $\iff \Phi_2 = K$ (exactly the attack set).

So, here are some consistency properties:

1. $\Phi_1=\text{True}$ only if $K \neq \{\}$. Monitor rings alarm only when/if there is actually an attack. Another way to write this is: $\Phi_1=\text{True} \implies K \neq \{\}$, there is an attack. We make sure there are no false positives in detection.
2. (Φ_1 and Φ_2 are consistent). $\Phi_1=\text{False} \iff \Phi_2 = \{\}$. This is the same as saying $\Phi_1=\text{True} \implies \Phi_2 \neq \{\}$.
3. $\Phi_2 = \hat{K} \subseteq \{1, 2, \dots, n_x + n_y\}$ only if there is no other set S , with same number or less variables, such that $y(t) = y(x_0, u_k, t) = y(x', u_s, t)$. When the monitor gives in output True, and a guess $\hat{K} = \{1, 3\}$, this means that it's not possible to find another potential attack set S which is simpler (two or less variables, in the example), such that something happens (seen later). So, $y(t)$ is the data coming from the system, $y(x_0, u_k, t)$ is the real explanation of data, $y(x', u_s, t)$ is an alternative explanation of data, with x' being another possible initial state, and u_s another possible attack working on a simpler or equally complex attack set. When my monitor gives me a guess for the variables under attack, if the monitor respects property 3, I am sure there is no simpler explanation for the data. If this is not the case, maybe by having $\hat{K}' = \{1, 3\}$ and $\hat{K}'' = \{3, 5\}$, then $\Phi_2 = \{1, 2, \dots, n_x + n_y\}$.